React.memo, useMemo, useCallback

1. React.memo - Skip Child Re-renders

Problem

```
function Parent() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <Child name="Rohit" />  {/* Re-renders every time! */}
      <button onClick={() => setCount(count + 1)}>Click: {cou
nt}</button>
    </div>
  );
}
```

Issue: Child re-renders when Parent re-renders, even though `name` prop never changes.

Solution

```
const Child = React.memo(function Child({ name }) {
  return <h2>Hello {name}</h2>;
});
```

Result: Child only re-renders when `name` prop actually changes.

How It Works

React.memo compares old props with new props using `===`:

- If same → Skip re-render
- If different → Re-render

The Gotcha - Objects Break It

```
const Child = React.memo(function Child({ user }) {
  return <h2>Hello {user.name}</h2>;
});

function Parent() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <Child user={{ name: "Rohit" }} />  {/* NEW object ever
y render! */}
      <button onClick={() => setCount(count + 1)}>Click</butt
on>
    </div>
  );
}
```

Problem: `{ name: "Rohit" }` creates new object every render.

```
{} === {}  // false (different memory addresses)
```

React.memo sees "different" object → Child re-renders every time.

This is where useMemo comes in.

---

2. useMemo - Cache Values & Keep Same Reference

Syntax

```
const cachedValue = useMemo(() => {
  return expensiveCalculation();
```

```
}, [dependencies]);
```

Problem 1: Objects Breaking React.memo

```
const Child = React.memo(function Child({ user }) {
  return <h2>Hello {user.name}</h2>;
});

function Parent() {
  const [count, setCount] = useState(0);

  // Fix: Cache the object
  const user = useMemo(() => {
    return { name: "Rohit" };
  }, []); // Empty array = create once, never recreate

  return (
    <div>
      <Child user={user} />
      <button onClick={() => setCount(count + 1)}>Click: {cou
nt}</button>
    </div>
  );
}
```

Result: Child doesn't re-render when count changes ✅

Problem 2: Expensive Calculations

```
function App() {
  const [count, setCount] = useState(0);
  const [numbers] = useState(Array(10000).fill(1));

  // Without useMemo: Calculates on EVERY render
  const sum = numbers.reduce((acc, n) => acc + n, 0);
```

```
  // With useMemo: Calculates once, caches result
  const memoizedSum = useMemo(() => {
    return numbers.reduce((acc, n) => acc + n, 0);
  }, [numbers]);

  return (
    <div>
      <p>Sum: {memoizedSum}</p>
      <button onClick={() => setCount(count + 1)}>Click: {cou
nt}</button>
    </div>
  );
}
```

When useMemo Runs

1. **First render** - Always runs
2. **When dependencies change** - Runs and caches new value
3. **When dependencies same** - Returns cached value (doesn't run)

```
const result = useMemo(() => {
  console.log('Running...');
  return a + b;
}, [a, b]);
```

- Change `a` or `b` → Logs "Running..."
- Change unrelated state → Nothing (returns cached value)

Empty Dependencies

```
const user = useMemo(() => ({ name: "Rohit" }), []);
```

Empty `[]` = Run once on mount, never again.

Common Bug: Missing Dependencies

```
function App() {
  const [multiplier, setMultiplier] = useState(2);

  // BUG! Missing dependency
  const result = useMemo(() => {
    return 10 * multiplier;
  }, []); // Should be [multiplier]

  // result always stays 20, never updates!
}
```

3. useCallback - Cache Functions

Problem

```
const Child = React.memo(function Child({ onClick }) {
  return <button onClick={onClick}>Click me</button>;
});

function Parent() {
  const [count, setCount] = useState(0);

  // NEW function every render
  const handleClick = () => {
    console.log('Clicked');
  };

  return (
    <div>
      <Child onClick={handleClick} />  {/* Child re-renders e
very time! */}
      <button onClick={() => setCount(count + 1)}>Click: {cou
```

```
nt}</button>
    </div>
  );
}
```

Problem: Function is recreated every render → Different reference → React.memo fails.

Solution

```
function Parent() {
  const [count, setCount] = useState(0);

  // Cache function, keep same reference
  const handleClick = useCallback(() => {
    console.log('Clicked');
  }, []);

  return (
    <div>
      <Child onClick={handleClick} />
      <button onClick={() => setCount(count + 1)}>Click: {cou
nt}</button>
    </div>
  );
}
```

Result: Child doesn't re-render when count changes ✅

Syntax

```
const cachedFunction = useCallback(() => {
  // function logic
}, [dependencies]);
```

useCallback = useMemo for Functions

```
// These are IDENTICAL:
const handleClick = useCallback(() => {
  console.log('Clicked');
}, []);

const handleClick = useMemo(() => {
  return () => {
    console.log('Clicked');
  };
}, []);
```

With Dependencies

```
function App() {
  const [count, setCount] = useState(0);
  const [name, setName] = useState("Rohit");

  const handleClick = useCallback(() => {
    console.log('Name is:', name);
  }, [name]); // Recreate when name changes

  // Changing count → Same function
  // Changing name → New function with updated name
}
```

Common Bug: Stale Closure

```
function App() {
  const [count, setCount] = useState(0);

  // BUG! Missing dependency
  const handleClick = useCallback(() => {
    console.log('Count is:', count);
```

```
  }, []); // Should be [count]

  // count = 5, but logs "Count is: 0" (stale!)
}
```

Safe Pattern with setState

```
function App() {
  const [todos, setTodos] = useState([]);

  // Safe: No dependencies needed
  const addTodo = useCallback((text) => {
    setTodos(prev => [...prev, { id: Date.now(), text }]);
  }, []); // Empty array is safe here
}
```

Why safe? We use functional update `prev =>` which always gets latest state.

When to Use (and NOT Use)

❌ DON'T Use For

Simple operations:

```
// DON'T wrap these
const sum = a + b;
const fullName = firstName + " " + lastName;
const double = n * 2;
```

Small arrays:

```
// DON'T wrap this
const filtered = [1,2,3,4,5].filter(n => n > 2);
```

Functions not passed to children:

```
// DON'T wrap this
const handleClick = () => setCount(count + 1);
```

✅ DO Use For

Large data processing:

```
// DO wrap this
const filtered = useMemo(() => {
  return students.filter(s => s.name.includes(search)); // 50
00+ items
}, [students, search]);
```

Expensive calculations:

```
// DO wrap this
const fibonacci = useMemo(() => {
  function fib(n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
  }
  return fib(30); // Millions of calculations
}, [n]);
```

Objects/Functions passed to React.memo:

```
// DO wrap these
const config = useMemo(() => ({ theme: 'dark' }), []);
const handleClick = useCallback(() => {}, []);

return <ExpensiveChild config={config} onClick={handleClick}
/>;
```

Sorting large arrays:

```
// DO wrap this
const sorted = useMemo(() => {
  return data.sort((a, b) => b.score - a.score); // 1000+ ite
ms
}, [data]);
```

Quick Reference Table

Hook	Purpose	Use For
React.memo	Skip child re-renders	Expensive child components
useMemo	Cache values	Objects/arrays for props, expensive calculations
useCallback	Cache functions	Functions passed to React.memo children

When they run:

- ✅ First render (always)
- ✅ When dependencies change
- ❌ When dependencies same (returns cached)

Complete Example

```
// Child with React.memo
const TodoItem = React.memo(function TodoItem({ todo, onToggl
e }) {
  return (
    <div onClick={() => onToggle(todo.id)}>
      {todo.text}
    </div>
  );
});

// Parent
function TodoList() {
```

```
  const [todos, setTodos] = useState([]);
  const [filter, setFilter] = useState('all');

  // useMemo: Cache filtered list
  const filteredTodos = useMemo(() => {
    return todos.filter(t =>
      filter === 'all' || t.status === filter
    );
  }, [todos, filter]);

  // useCallback: Cache function
  const handleToggle = useCallback((id) => {
    setTodos(prev =>
      prev.map(t => t.id === id ? { ...t, done: !t.done } :
t)
    );
  }, []);

  return (
    <div>
      {filteredTodos.map(todo => (
        <TodoItem
          key={todo.id}
          todo={todo}
          onToggle={handleToggle}
        />
      ))}
    </div>
  );
}
```

Why this works:

1. `filteredTodos` only recalculates when `todos` or `filter` changes
2. `handleToggle` keeps same reference across renders

3. `TodoItem` only re-renders when its `todo` data changes

Golden Rule

> "Profile first, optimize later"

1. Write simple code first
2. Use React DevTools Profiler to find slow components
3. Optimize only what's actually slow (> 16ms)
4. Don't optimize prematurely

Most apps don't need these hooks everywhere!

\